

ECLIPSE FOUNDATION • ECLIPSE THREADX

Eclipse ThreadX

Getting Started

A Modern Board Enablement Framework

Google Summer of Code 2026 – Project Proposal

Applicant	Ali Eissa
University	The British University in Dubai
Email	ali.eissa.dev@gmail.com
GitHub	github.com/alieissa-commits
Mentor	Frédéric Desbiens
Project Size	350 hours (Large)

Contents

Contents	2
Executive Summary	4
1.1 Problem Statement	4
1.2 Proposed Solution	4
1.3 Key Deliverables	5
1.4 Alignment with Project Goals	5
Project Analysis	6
2.1 Architectural Foundation: Migrating to SampleX	6
2.1.1 Architectural Strengths	6
2.2 Gap Analysis	7
2.3 Requirements Traceability	7
Technical Architecture	8
3.1 Repository Design Structure	8
3.2 Build System Architecture	9
3.3 Board Abstraction Pattern	9
3.4 Component Integration Map	10
Board Selection & Rationale	11
4.1 Selection Criteria	11
4.2 Selected Boards	11
Use Case Design	12
5.1 UC1: Network-Connected Environmental Station	12
5.1.1 Scenario	12
5.1.2 ThreadX Architecture	12
5.1.3 Components Showcased	12
5.2 UC2: RISC-V Telemetry Edge Node	13
5.2.1 Scenario	13
5.2.2 ThreadX Architecture	13
5.2.3 Key Value	13
Implementation Plan & Timeline	14
6.1 Phased Approach	14
6.2 Detailed Week-by-Week Plan	14

6.2.1 Phase 1: Foundation (Weeks 1-3)	14
6.2.2 Phase 2: STM32 Demos (Weeks 4-6).....	14
6.2.3 Phase 3: RISC-V/ESP32-C6 (Weeks 7-9).....	15
6.2.4 Phase 4: Documentation & Polish (Weeks 10-12).....	15
Testing & Quality Assurance.....	16
7.1 Build Verification.....	16
7.2 Functional Testing.....	16
7.3 Proposed CI Pipeline.....	16
Documentation Strategy.....	17
8.1 Board Addition Guide.....	17
8.2 SDK Update Procedures	17
8.3 Per-Board Documentation.....	17
Risk Assessment	18
About the Applicant.....	19
10.1 Technical Background.....	19
10.2 Motivation.....	19
10.3 Availability.....	19
References.....	20

Executive Summary

1.1 Problem Statement

Eclipse ThreadX powers over 12 billion devices worldwide, yet its developer onboarding experience has not kept pace with the project's transition from Microsoft Azure RTOS to the Eclipse Foundation. The legacy Getting Started repository remains tightly coupled to Azure IoT services, while the clean architecture introduced in the `iot-devkit` repository (now being transitioned to the official sample repository) currently covers only the MXChip AZ3166 board. This leaves developers targeting other silicon vendors and modern architectures without a clear, modern entry point to learn the RTOS.

1.2 Proposed Solution

This proposal presents a **board enablement framework** for the new sample repository, migrating and extending clean architecture originally introduced in `iot-devkit` to support two distinct, high-value evaluation platforms: the *STMicroelectronics STM32F767ZI-Nucleo-144 (ARM Cortex-M7)* and the *ESP32-C6-DevKitC-1-N8 (RISC-V)*. Rather than creating isolated samples, the design establishes a **repeatable pattern** for community contributors to add new hardware with minimal friction.

Each board ships with a use-case-driven demo application that exercises real ThreadX RTOS primitives (threads, message queues, event flags, timers) alongside meaningful hardware peripherals—moving far beyond bare-bones UART printf loops.

1.3 Key Deliverables

Deliverable	Description
Board Support Packages	Complete BSPs for 2 hardware evaluation boards (1× STMicroelectronics Cortex-M7 + 1× Espressif RISC-V).
Demo Applications	Use-case-driven, multi-threaded demos for each board exercising specific middleware components.
Documentation	Board addition guide, SDK update procedures, and per-board README.adoc reference files.
Build Infrastructure	CMake presets, vendor IDE compatibility notes (STM32CubeIDE, ESP-IDF integration), and a CI pipeline prototype.

1.4 Alignment with Project Goals

- **STMicroelectronics Priority & Advanced Middleware:** ST is a vendor actively contributing code to Eclipse ThreadX. Validating the Cortex-M7 toolchain on the STM32F767ZI solidifies support for high-performance ARM targets.
- **RISC-V Interest:** The ESP32-C6 validates ThreadX's RISC-V port natively, addressing the ecosystem's massive pivot toward RISC-V and tapping into Espressif's massive developer community.
- **Architectural Scalability:** Successfully supporting both a traditional bare-metal ARM target and a complex RISC-V SoC proves the extreme generality and robustness of the proposed board abstraction framework.
- **Use-Case Thinking:** Demos map to real-world domains (environmental monitoring, wireless telemetry nodes) rather than pure technology showcases.

Project Analysis

2.1 Architectural Foundation: Migrating to SampleX

The `iot-devkit` repository demonstrated a clean, well-structured architecture. This project will migrate this architectural pattern into the official `samplex` repository, transforming it from a single-target solution (MXChip AZ3166) into a robust, multi-architecture framework.

- **Build System:** CMake with Ninja generator, ARM GCC cross-compilation toolchain, preset-based configuration..
- **Library Management:** Integrates ThreadX and NetX Duo as Git submodules, employing CMake OBJECT libraries to handle HAL drivers and hardware sensors.
- **Shared utilities:** Cross-platform `jsmn` (JSON), `nanoprintf`, and `newlib_nano` stubs under `shared`.
- **Board Abstraction:** Enforces a strict separation of concerns between raw startup code, board initialization sequences, and high-level application logic.

2.1.1 Architectural Strengths

1. **Toolchain Abstraction:** The `cmake/arm-gcc-cortex-m4.cmake` $\rightarrow$ `arm-gcc-cortex-toolchain.cmake` hierarchy allows different Cortex-M cores to share the generic toolchain logic.
2. **Find Module Pattern:** The `FindSTM32HAL.cmake` and `FindCMSIS.cmake` modules successfully encapsulate vendor SDK discovery, making STM32 integration highly declarative and modular.
3. **Post-build utilities:** The `utilities.cmake` module provides portable `post_build()` and `set_target_linker()` functions
4. **Object library pattern:** Core libraries use the CMake OBJECT type for fine-grained linking control on memory-constrained targets.

2.2 Gap Analysis

Gap	Impact
Single board (AZ3166 only)	Developers with other boards have no starting point to build from.
No RISC-V or Espressif support	The framework ignores a massive demographic of IoT developers using low-cost Wi-Fi/Thread SoCs.
No NetX Duo / FileX demos	Key Eclipse ThreadX middleware components are missing from modern board contexts.
No CI pipeline	Build regressions go unnoticed.

2.3 Requirements Traceability

Project Requirement	Proposed Deliverable
Clean, empty starter applications	Board skeleton templates with minimal ThreadX boot logic.
Samples leveraging hardware features	Use-case-driven demos utilizing Ethernet, Wi-Fi/Bluetooth stacks (where applicable), and I2C/SPI sensors.
Architectural diversity	One Cortex-M7 board (STM32F767ZI) and one RV32IMAC board (ESP32-C6).
Documentation	Step-by-step guide for SDK version bumps and board additions.

Technical Architecture

3.1 Repository Design Structure

The proposed structure follows the <Vendor>/<Board>/ convention established by the MXChip target, enabling intuitive navigation while accommodating the ESP-IDF component structure.

```
1  samplex/
2      cmake/                                # Shared toolchain files
3          arm-gcc-cortex-m4.cmake
4          arm-gcc-cortex-m7.cmake          # NEW : Cortex - M7 variant
5          arm-gcc-cortex-toolchain.cmake
6          riscv-gcc-toolchain.cmake
7          utilities.cmake
8      shared/                               # Cross - board shared code
9          lib/      (threadx, netxduo, filex, jsmn, nanoprintf)
10         src/      (newlib_nano.c, app_common)
11  MXChip/
12      AZ3166 /                             # EXISTING
13  STMicroelectronics/                     # NEW VENDOR DIRECTORY
14      STM32F767ZI-Nucleo/                 # Deliverable 1
15  Espressif/                             # NEW VENDOR DIRECTORY
16      ESP32-C6-DevKitC-1/                 # Deliverable 2
```


3.2 Build System Architecture

Each board will utilize **CMake Presets**. This allows a developer to build for a specific target with a single command (e.g., `cmake --preset stm32f767-nucleo`), abstracting away the complexity of cross-compilation flags.

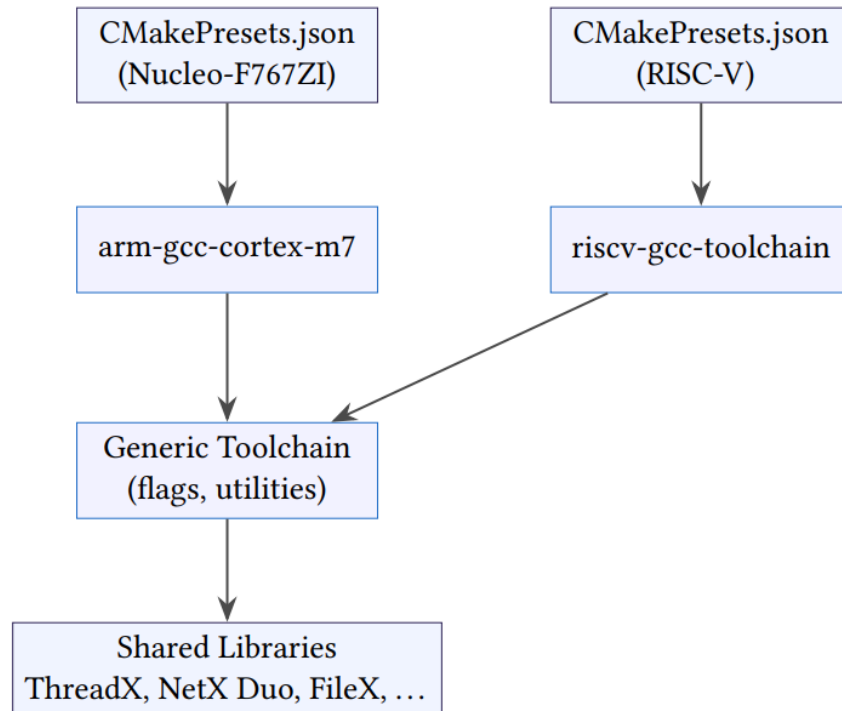


Figure 3.1: CMake toolchain hierarchy — new boards only require a thin preset layer.

3.3 Board Abstraction Pattern

Each board follows a strict separation of concerns

Layer	Contents
startup/	Vendor-provided: vector table, linker script, SystemInit, ThreadX lowlevel init
board_init.c/h	Board-specific: clock configuration, GPIO pins, peripheral initialization
lib/stm32cubef7/	HAL driver objects with board-specific <code>stm32f7xx_hal_conf.h</code>
main.c	Application logic: ThreadX threads, demo behavior
console.c	UART ↔ printf redirection

3.4 Component Integration Map

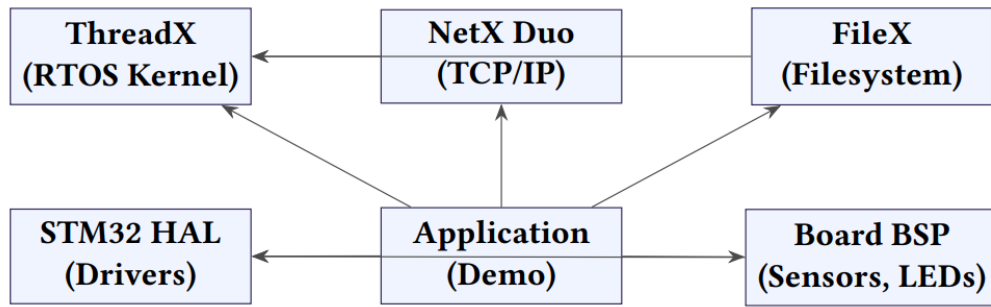


Figure 3.2:

- **Board 1 (STM32F7):** Integrates ThreadX (Kernel), NetX Duo (TCP/IP), FileX (Filesystem), and STM32 HAL.
- **Board 2 (ESP32-C6):** Integrates ThreadX (Kernel) natively on RISC-V, interfacing with ESP-IDF peripheral drivers.

Board Selection & Rationale

4.1 Selection Criteria

Board selection was driven by five factors:

1. **Vendor alignment:** Expanding STMicroelectronics coverage while tapping into RISC-V's massive community.
2. **Peripheral diversity:** Boards must support non-trivial networking/storage demos.
3. **Architecture validation:** Validating both advanced ARM (Cortex-M7) and RISC-V (RV32IMAC).
4. **Community accessibility:** Both boards are globally available, well-documented, and highly affordable.

4.2 Selected Boards

Board	Core	Peripherals	Rationale
STM32F767ZI-Nucleo-144	Cortex-M7 (216 MHz)	Ethernet, Arduino headers, USB OTG	ST priority; Ethernet enables powerful NetX Duo networking demos; validates Cortex-M7 toolchain.
ESP32-C6-DevKitC-1-N8	RV32IMAC (160 MHz)	Wi-Fi 6, Bluetooth 5, 802.15.4, SPI, UART	Validates ThreadX's RISC-V port natively; bridges Eclipse ThreadX with the dominant modern IoT SoC family.

Use Case Design

“Rather than thinking in pure technological terms, I would like students to think in terms of use cases.” — Frédéric Desbiens

5.1 UC1: Network-Connected Environmental Station

Board: STM32F767ZI-Nucleo-144 **Domain:** Building environment monitoring / smart agriculture

5.1.1 Scenario

An environmental monitoring station that collects temperature and humidity data from sensors connected via I2C (using the Arduino shield headers), reports readings over Ethernet using MQTT, and logs data locally to a **FileX RAM disk**.

5.1.2 ThreadX Architecture

Thread	Priority	Responsibility
sensor_thread	3	Reads I2C sensors periodically; publishes to message queue.
network_thread	4	Consumes sensor data; transmits via NetX Duo MQTT client.
logger_thread	5	Writes buffered data to RAM disk via FileX.
status_thread	6	Blinks LEDs to indicate system health and Ethernet link status.

5.1.3 Components Showcased

- **NetX Duo:** Ethernet driver, DHCP client, DNS client, MQTT publish.
- **FileX:** FAT filesystem on a RAM disk, demonstrating file lifecycle and buffered writes without requiring external storage hardware.
- **ThreadX:** Four cooperating threads using message queues, semaphores, and byte pools.

5.2 UC2: RISC-V Telemetry Edge Node

Board: ESP32-C6-DevKitC-1-N8 **Domain:** IoT Edge Computing / Telemetry

5.2.1 Scenario

A lightweight telemetry data acquisition node leveraging the native RISC-V architecture. It periodically collects simulated or physical sensor readings via UART/SPI, safely buffers them in RAM using ThreadX byte pools, and provides a UART Command Line Interface (CLI) for configuration and data readback.

5.2.2 ThreadX Architecture

Thread	Priority	Responsibility
acquisition_thread	3	Periodic data collection from UART/SPI sensor feed.
processing_thread	4	Buffered data processing using ThreadX RAM-backed pools
cli_thread	5	UART command interface for configuration and readback.

5.2.3 Key Value

- Validates Eclipse ThreadX's RISC-V port (RV32IMAC) in a real application context.
- Demonstrates integration with Espressif's ESP-IDF, paving the way for advanced Wi-Fi/Thread implementations.
- Exercises ThreadX TX_TIMER for periodic scheduling and TX_BYTE_POOL for dynamic memory.

Implementation Plan & Timeline

6.1 Phased Approach

The 350-hour project is divided into four phases with clear milestones:

Phase	Weeks	Hours	Focus
Foundation	1-3	~90	Environment, architecture design, STM32F7 BSP skeleton + Clocks.
STM32 Demo	4-6	~90	Full demos for STM32F7: NetX Duo, MQTT, and FileX SD logging.
RISC-V/ESP32	7-9	~100	ESP-IDF toolchain integration, RV32IMAC port, Edge Node Demo.
Polish	10-12	~70	Documentation, CI pipeline, IDE integration testing, Final PRs.

6.2 Detailed Week-by-Week Plan

6.2.1 Phase 1: Foundation (Weeks 1-3)

- **Week 1:** Development environment validation. Migrate the base `iot-devkit` CMake architecture into the `samplex` repository and validate the existing `MXChip AZ3166` build. Install ESP-IDF toolchain alongside ARM GCC.
- **Week 2:** Create `arm-gcc-cortex-m7.cmake`. STM32F767ZI board skeleton: linker script, `SystemClock_Config` (216 MHz), Ethernet PHY init (LAN8742A).
- **Week 3:** Validate ThreadX boot on STM32F7: single thread blinks LEDs.

6.2.2 Phase 2: STM32 Demos (Weeks 4-6)

- **Week 4:** NetX Duo integration: IP stack, DHCP client, TCP echo server test. Validate Ethernet connectivity.
- **Week 5:** Environmental station demo: I2C sensor reads, MQTT publish via NetX Duo.
- **Week 6:** FileX SD card logging integration. Demo polishing.

6.2.3 Phase 3: RISC-V/ESP32-C6 (Weeks 7-9)

- **Week 7:** Create `riscv-gcc-toolchain.cmake`. Integrate ESP-IDF CMake system. Port ThreadX boot for RV32IMAC.
- **Week 8:** Basic UART and SPI I/O on the ESP32-C6 using ThreadX primitives.
- **Week 9:** RISC-V edge data logger demo completion: periodic acquisition, RAM buffering, CLI interface.

6.2.4 Phase 4: Documentation & Polish (Weeks 10-12)

- **Week 10:** Board addition guide and SDK update procedures written.
- **Week 11:** GitHub Actions CI prototype (ARM GCC + RISC-V automated build verification).
- **Week 12:** Cross-board regression testing. Code cleanup, final PR submissions.

Testing & Quality Assurance

7.1 Build Verification

All code will compile cleanly with `-Wall -Werror` under ARM GCC and RISC-V GCC (required) and be validated against C99 compliance via `CMAKE_C_STANDARD 99`. Binary sizes will be tracked to ensure they fit within target Flash limits.

7.2 Functional Testing

For contributors with physical boards, manual testing protocols will include:

- Flash binary, observe LED patterns matching expected behavior.
- Verify serial console output at 115200 baud.
- For networking demos: verify DHCP address acquisition and MQTT publish via Wireshark.

7.3 Proposed CI Pipeline

```
1  name: Build Verification
2  on: [push, pull_request]
3  jobs:
4      build-arm:
5          runs-on: ubuntu-latest
6          steps:
7              - uses: actions/checkout@v4
8              - run: sudo apt-get install -y gcc-arm-none-eabi
9              - run: cd src/STMicroelectronics/STM32F767ZI-Nucleo
10             && ./scripts/build.sh
11      build-riscv:
12          runs-on: ubuntu-latest
13          steps:
14              - uses: actions/checkout@v4
15              - name: Setup ESP-IDF
16              - uses: espressif/esp-idf-ci-action@v1
17              - run: cd src/Espressif/ESP32-C6-DevKitC-1 &&
idf.py build
```

Documentation Strategy

8.1 Board Addition Guide

A step-by-step guide enabling community contributors to add a new board in under a day:

1. **Create directory:** <vendor>/<Board> with standard subdirectories
2. **Toolchain:** Select or create appropriate `cmake/arm-gcc-cortex-*.cmake`
3. **Startup files:** Port from vendor SDK: linker script, vector table, `SystemInit`, `tx_initialize_low_level.S`
4. **HAL configuration:** Create `stm32f7xx_hal_conf.h` with required module enables
5. **Board init:** Write `board_init.c` with clock tree, GPIO, and peripheral setup
6. **Application:** Create `main.c` with ThreadX thread(s) exercising board peripherals
7. **Build & test:** Verify build with `-Wall -Werror`, test on hardware.

8.2 SDK Update Procedures

When vendors release new SDK versions (e.g., STM32CubeF7 or ESP-IDF bumps):

1. Download new SDK package from vendor website
2. Replace `Drivers/CMSIS/` and `Drivers/STM32F7xx_HAL_Driver/` directories
3. Verify `FindCMSIS.cmake` and `FindSTM32HAL.cmake` still locate all headers
4. Check for HAL API changes in release notes; update `board_init.c` if needed
5. Rebuild all affected boards; verify zero warnings
6. Update version reference in board `README.adoc`

8.3 Per-Board Documentation

Each board `README.adoc` will include:

- Board overview with key specifications
- Demo description with architecture diagram
- Prerequisites (toolchain, tools)
- Build instructions (script and manual CMake)
- Flash / deploy instructions
- Serial console setup
- Project directory structure reference

Risk Assessment

Risk	Prob.	Impact	Mitigation
ESP-IDF / FreeRTOS Coupling	High	High	Leverage the ESP-IDF OS wrapper layer (OSAL) to attempt mapping underlying FreeRTOS calls to ThreadX primitives. If Espressif's Wi-Fi binary blobs still prove too deeply coupled to bypass efficiently within the timeline, fall back to the basic Telemetry Edge Node demo (using bare-metal UART/SPI) to ensure successful native RISC-V validation.
Hardware & Peripheral Complexity	Med	Med	Start with a proven TCP echo server before attempting the more complex MQTT protocol. By utilizing a FileX RAM disk for storage, we completely mitigate the hardware risks associated with physical SDIO wiring and external card compatibility.
Toolchain Integration Delays	Low	Med	Dedicate specific time in Week 7 to isolate the RISC-V toolchain setup. Build simple bare-metal examples first to ensure the compiler works before integrating the ThreadX RTOS.

About the Applicant

10.1 Technical Background

I am an undergraduate student with a strong foundation in systems programming and embedded development:

- **Languages:** Proficient in C (C99), comfortable with C++.
- **Build systems:** Experienced with CMake, Make, and Ninja.
- **Networking:** Familiar with TCP/IP fundamentals, MQTT, and HTTP protocols.
- **Software Engineering:** Experienced in building complex, decoupled frameworks.
- **Algorithms:** Able to demonstrate optimization skills needed for embedded systems.

10.2 Motivation

Eclipse ThreadX's unique position as the only open-source RTOS certified for safety-critical applications makes it critically important for the embedded industry. The fact that it runs in 12+ billion devices yet lacks modern Getting Started resources for the most ubiquitous modern boards (like STM32 Nucleo and ESP32 series) represents a critical gap. I want to contribute by building samples that genuinely help developers working on modern IoT edge nodes. I aim to not only learn from this journey but also provide meaningful value.

10.3 Availability

I am available to dedicate 25-30 hours per week throughout the 12-week coding period.

References

1. ThreadX Documentation Set: <https://github.com/eclipse-threadx/rtos-docs-asciidoc>
2. SampleX Repository: <https://github.com/eclipse-threadx/samplex>
3. ThreadX GitHub Organization: <https://github.com/eclipse-threadx>
4. ThreadX (RTOS Kernel): <https://github.com/eclipse-threadx/threadx>
5. ThreadX NetX Duo: <https://github.com/eclipse-threadx/netxdxduo>
6. ThreadX FileX: <https://github.com/eclipse-threadx/filex>
7. STM32 Nucleo-144 F767ZI: <https://www.st.com/en/evaluation-tools/nucleo-f767zi.html>
8. Espressif ESP32-C6-DevKitC-1: <https://docs.espressif.com/projects/espressif-esp-dev-kits/en/latest/esp32c6/esp32-c6-devkitc-1/index.html>
9. ESP-IDF Programming Guide: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/>